

Gesture Recognition Module (T4.3)

Contact info: Foteinos Konstantinos (kfoteinos@hua.gr)

Setup

This process should have already been done.

1. Clone the repository: `git clone https://github.com/foteinosKonstantinos/gesture_module_v2`
2. Add execution privileges: `chmod +x run_producer.sh setup_venv.sh compile_and_run.sh`
3. Setup the python virtual environment: `./setup_venv.sh`
4. Change absolute paths (if needed): weights (lines 908-909 in `classifier.py`, the YOLO pose wrapper downloads the weights automatically), the `executable` property (file `setup.cfg`), which is the path to the python virtual environment, and the `PATH` constant (`producer.py`, line 13).

Run the module

The initial configuration of the module uses a dummy producer node to generate simulation data.

1. Compile and run the node (any changes will be compiled automatically):
`./compile_and_run.sh`
2. Run the producer (only during testing): `./run_producer.sh`
3. Inspect the logs (recommended) or print the `gesture_command` topic: `ros2 topic echo /gesture_command --once --full`

Configure the module

1. **Classification model:** The gesture recognition module (`classifier.py`) supports two classifiers: EfficientNet-B0 and YOLO26m-class, pretrained on the FR-GESTURE (extended with an additional Stop class). To choose between the two, change lines 909 and 910 (`classifier=` , comment out the one model that you don't want to use and uncomment the other assignment):

```

rclpy.spin(node=Gesture_Commander_Coordinator(
    classifier = EfficientNetB0_Wrapper(config=config,path="/home/triffid/
    # classifier = YOLO_Classification_Wrapper(config=config,path="/home/
    pose_estimator = YOLO_Pose_Wrapper(model="yolo26n-pose.pt", config=cc
    perceptron = DEMO_Perceptron(),
    # perceptron = RealSense_Perceptron(),
    config = config,
    transformations = Approximate_Transformations(config=config)
))

```

2. **Topic names:** On lines 879-884 (on the same script), change the topic names if necessary (probably, you just need to remove the `_test` postfix):

```

nav_fix_topic = "/fix_test",
odom_topic = "/dog_odom_test",
depth_topic = "/b2/camera_front_435i/realsense_front_435i/depth/image_rec
rgb_topic = "/b2/camera_front_435i/realsense_front_435i/color/image_raw_t
camera_info = "/b2/camera_front_435i/realsense_front_435i/color/camera_ir
output_topic = "/gesture_command",

```

3. **Action servers:** If any changes are made to the action servers (trigger names, etc), please contact kfoteinos@hua.gr to proceed with the necessary modifications in the code. Changing only lines 897-907 may not be enough.
4. **Classification rate:** To change the maximum FPS, modify the value of `max_classification_rate` (line 894).
5. **Successive frames:** To change the number of consecutive frames required to activate triggers, change the `min_occurrences` parameter (line 890) - larger values make the model more stringent and reduce false positives.
6. **Time limit between triggers:** To change the required time between two trigger activations, modify the `min_sec_between_commands` parameter (line 905) - expressed in seconds, larger values lead to fewer action calls and increase robustness to false positives.
7. Follow the previous steps to compile and run the package.